TUGAS FORMATIF 2

PEMROGRAMAN MOBILE LANJUT

Disusun oleh:

M. Rizky Gedsha Pratama 2022320065

PROGRAM STUDI SISTEM INFORMASI

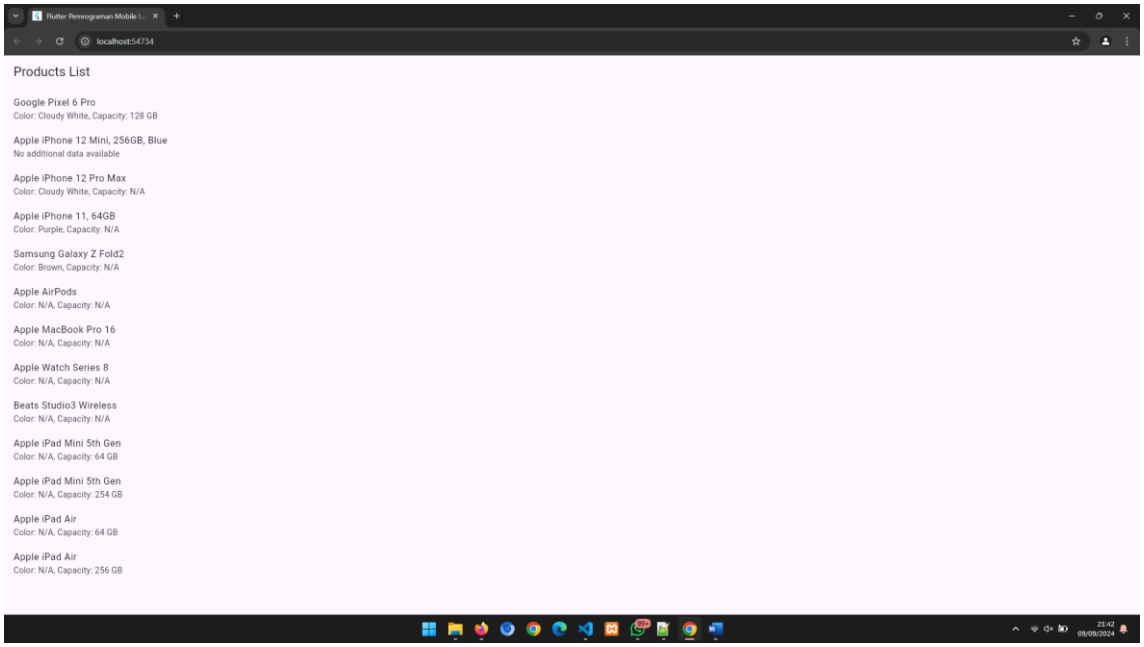
FAKULTAS INFORMATIKA

UNIVERSITAS BINA INSANI

BEKASI

2024

OUTPUT



main.dart

```dart
import 'package:flutter/material.dart';
import 'models/data_model.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Flutter Pemrograman Mobile Lanjut',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        useMaterial3: true,
      ),
      home: const FormatifRizky(),
    );
  }
}
```

data_model.dart

```dart
// ignore_for_file: library_private_types_in_public_api

import 'package:flutter/material.dart';
import '../services/services.dart';

class FormatifRizky extends StatefulWidget {
  const FormatifRizky({super.key});

  @override
  _FormatifRizkyState createState() => _FormatifRizkyState();
}

class _FormatifRizkyState extends State<FormatifRizky> {
  late Future<List<Product>> futureProducts;

  @override
  void initState() {
    super.initState();
    futureProducts = ProductService().fetchProducts(); // Memanggil fetch
data
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Products List'),
      ),
      body: FutureBuilder<List<Product>>(
        future: futureProducts,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          } else if (snapshot.hasError) {
            return Center(child: Text('Error: ${snapshot.error}'));
          } else if (snapshot.hasData) {
            List<Product> products = snapshot.data!;

            return ListView.builder(
              itemCount: products.length,
              itemBuilder: (context, index) {
                final product = products[index];
                final data = product.data;

                return ListTile(
```

```
                  title: Text(product.name),
                  subtitle: data != null
                      ? Text(
                          'Color: ${data['color'] ?? 'N/A'}, Capacity:
${data['capacity'] ?? data['Capacity'] ?? 'N/A'}')
                      : const Text('No additional data available'),
                );
              },
            );
          } else {
            return const Center(child: Text('No products found.'));
          }
        },
      ),
    );
  }
}
```

services.dart

```
// ignore_for_file: depend_on_referenced_packages

import 'dart:convert';
import 'package:http/http.dart' as http;

class Product {
  final String id;
  final String name;
  final Map<String, dynamic>? data;

  Product({required this.id, required this.name, this.data});

  // Factory method untuk mem-parsing JSON
  factory Product.fromJson(Map<String, dynamic> json) {
    return Product(
      id: json['id'],
      name: json['name'],
      data: json['data'],
    );
  }
}

class ProductService {
  // Ganti dengan URL API-mu
  final String apiUrl = 'https://api.restful-api.dev/objects';
```

```
  // Metode untuk fetch data dari API
  Future<List<Product>> fetchProducts() async {
    final response = await http.get(Uri.parse(apiUrl));

    if (response.statusCode == 200) {
      List<dynamic> body = jsonDecode(response.body);
      List<Product> products =
          body.map((dynamic item) => Product.fromJson(item)).toList();
      return products;
    } else {
      throw Exception('Failed to load products');
    }
  }
}
```

pubspec.yaml

```
name: pmlbiu
description: "A new Flutter project."
# The following line prevents the package from being accidentally
published to
# pub.dev using `flutter pub publish`. This is preferred for private
packages.
publish_to: "none" # Remove this line if you wish to publish to pub.dev

# The following defines the version and build number for your
application.
# A version number is three numbers separated by dots, like 1.2.43
# followed by an optional build number separated by a +.
# Both the version and the builder number may be overridden in flutter
# build by specifying --build-name and --build-number, respectively.
# In Android, build-name is used as versionName while build-number used
as versionCode.
# Read more about Android versioning at
https://developer.android.com/studio/publish/versioning
# In iOS, build-name is used as CFBundleShortVersionString while build-
number is used as CFBundleVersion.
# Read more about iOS versioning at
#
https://developer.apple.com/library/archive/documentation/General/Referen
ce/InfoPlistKeyReference/Articles/CoreFoundationKeys.html
# In Windows, build-name is used as the major, minor, and patch parts
# of the product and file versions while build-number is used as the
build suffix.
version: 1.0.0+1
```

```yaml
environment:
  sdk: ">=3.2.3 <4.0.0"

# Dependencies specify other packages that your package needs in order to
work.
# To automatically upgrade your package dependencies to the latest
versions
# consider running `flutter pub upgrade --major-versions`. Alternatively,
# dependencies can be manually updated by changing the version numbers
below to
# the latest version available on pub.dev. To see which dependencies have
newer
# versions available, run `flutter pub outdated`.
dependencies:
  flutter:
    sdk: flutter

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.2

dev_dependencies:
  flutter_test:
    sdk: flutter
  http: ^0.13.4

  # The "flutter_lints" package below contains a set of recommended lints
to
  # encourage good coding practices. The lint set provided by the package
is
  # activated in the `analysis_options.yaml` file located at the root of
your
  # package. See that file for information about deactivating specific
lint
  # rules and activating additional ones.
  flutter_lints: ^2.0.0

# For information on the generic Dart part of this file, see the
# following page: https://dart.dev/tools/pub/pubspec

# The following section is specific to Flutter packages.
flutter:
  # The following line ensures that the Material Icons font is
  # included with your application, so that you can use the icons in
  # the material Icons class.
  uses-material-design: true
```

```
  # To add assets to your application, add an assets section, like this:
  # assets:
  #   - images/a_dot_burr.jpeg
  #   - images/a_dot_ham.jpeg

  # An image asset can refer to one or more resolution-specific
"variants", see
  # https://flutter.dev/assets-and-images/#resolution-aware

  # For details regarding adding assets from package dependencies, see
  # https://flutter.dev/assets-and-images/#from-packages

  # To add custom fonts to your application, add a fonts section here,
  # in this "flutter" section. Each entry in this list should have a
  # "family" key with the font family name, and a "fonts" key with a
  # list giving the asset and other descriptors for the font. For
  # example:
  # fonts:
  #   - family: Schyler
  #     fonts:
  #       - asset: fonts/Schyler-Regular.ttf
  #       - asset: fonts/Schyler-Italic.ttf
  #         style: italic
  #   - family: Trajan Pro
  #     fonts:
  #       - asset: fonts/TrajanPro.ttf
  #       - asset: fonts/TrajanPro_Bold.ttf
  #         weight: 700
  #
  # For details regarding fonts from package dependencies,
  # see https://flutter.dev/custom-fonts/#from-packages
```